PART V — TOKENS, CONTEXT, AND MODEL CHOICE

Chapter 23

The Token Budget — What Actually Fits in Your Context Window

"A context window does not fill with meaning. It fills with tokens, and every one of them was paid for by something else that could have been there instead."

Chapter 23 — The Token Budget — What Actually Fits in Your Context Window

Every chapter so far has treated the LLM call as if it simply happens — a query goes in, a response comes out. Underneath every one of those calls sits a hard, finite number: the context window. Part IV built an agent that iterates, retrieves, compresses, drafts, and judges, and every one of those steps either reads from or writes into that same shared, shrinking budget. Part V starts by making the budget itself visible.

This chapter grounds the abstract idea of a "context window" in the real numbers this project has actually measured: `llama-3.1-8b-instant`'s 128,000-token window, the real per-iteration token counts a dry run logged, and the real bug — a 40-iteration runaway loop that blew through Groq's rate limit — that made token budgeting a first-class design concern rather than an afterthought.

23.1 What a context window is and what it includes

> **Definition — Context window**
> The total number of tokens a model can process in a single call, shared between everything sent to it (system prompt, conversation history, tool results, the current user message) and everything it is permitted to generate back.

A context window is not a private allowance for the user's question. It is one shared pool that every part of a single LLM call draws from: the system prompt (`_ROLE_AND_RULES` plus `_PROCESS_INSTRUCTIONS`, Chapter 26), the full conversation history accumulated so far (every prior assistant turn, every tool call, every tool result), the current user message, and the space reserved for the model's own response. None of these is free. A system prompt that grows by 500 tokens is 500 tokens the conversation history and the model's answer no longer have.

This is why Chapter 22's compression pipeline and this chapter are really the same story told from two directions. NAC, DC, and LBC exist to shrink what goes *into* the window before it is sent. This chapter is about what the window *is* — the fixed container those compression stages are shrinking content to fit inside.

It is also worth being precise about *when* the window is consumed. The context window is not a single running total drained gradually over a session the way a bank balance is — it is re-evaluated fresh on every individual API call. Each call to `llm`, `merge_llm`, or `judge_llm` sends its own complete prompt (system prompt plus however much history has accumulated up to that point) and must fit that entire prompt inside the window on its own. A run that has already made four LLM calls does not have a smaller window on its fifth call — it has the same 128,000-token window, now filled with more history than before, competing against the same fixed output reservation.

`_serialize_messages()` in `agent_query.py` truncates every message's logged content to 600 characters before writing it to the debug log, appending an ellipsis when truncation occurs. This is not a token-budget mechanism for the LLM call itself — the log line never goes near the model — but it is worth noticing as the same discipline applied one layer downstream: even a human-facing debug trace of a multi-iteration run needs its own size discipline, or a single verbose tool result can make the log as unreadable as an unbounded prompt would make the model's actual context.

23.2 Total window vs. usable input

ADR-005 records the two Groq models this project evaluated: `llama-3.1-8b-instant` at 128,000 total tokens with an 8,192-token output ceiling, and `llama-3.3-70b-versatile` at 128,000 total tokens with a 32,768-token output ceiling. Both numbers matter, and they are not the same number. The 128K figure is the *total* window — input plus output combined, in the way most providers document it. The output ceiling is a hard subtraction from that total before a single token of actual conversation history gets to occupy the remainder.

For the 8B model, that leaves roughly 120,000 tokens of genuinely usable input space after reserving room for the model's own response — and that number still assumes the full 8,192-token output ceiling is actually needed, which most turns in this pipeline do not require. `_PROCESS_INSTRUCTIONS` caps the final answer at 400 words specifically, so the model never needs to reserve anywhere near its full output ceiling, which is itself a token-budget decision as much as a formatting one.

Model	Total window	Output ceiling	Usable input (approx.)
llama-3.1-8b-instant	128,000	8,192	~119,800
llama-3.3-70b-versatile	128,000	32,768	~95,200

The 70B model's larger output ceiling shrinks its usable input budget relative to the 8B model, at the same total window size. A model swap is never a pure upgrade to the numbers in this table — it trades one constraint for another, and Chapter 25 returns to what the 70B model buys back in exchange.

23.3 A worked token budget for a 6-iteration agentic RAG run

A real dry-run trace (`Runs/RAG Dry Run (3).txt`) logs a `[CTXSIZE]` line after every LLM call, in the exact format `iter=N msgs=M chars=C prompt_tokens=P cum_prompt_tokens=T`. Iteration 1 opens at 820 prompt tokens with two messages. Iteration 2, after one round of `retrieve_documents` results accumulate into history, jumps to 1,724 tokens for that single call — cumulative 2,544. Iteration 3 adds another 1,947 — cumulative 4,491.

```
[CTXSIZE]       iter=1       msgs=2       chars=1959       prompt_tokens=820
cum_prompt_tokens=820
[CTXSIZE]       iter=2       msgs=6       chars=6389       prompt_tokens=1724
cum_prompt_tokens=2544
[CTXSIZE]       iter=3       msgs=8       chars=6507       prompt_tokens=1947
cum_prompt_tokens=4491
```

Notice that `cum_prompt_tokens` is a running sum of every prompt sent, not the size of any single prompt — Groq bills and rate-limits by tokens sent per call, so both numbers matter for different reasons: the per-call `prompt_tokens` figure determines whether any single call fits the window, while `cum_prompt_tokens` across a session determines whether the run stays under a rate-limit ceiling before it finishes. This trace's growth rate — roughly 1,700-1,950 tokens added per iteration once retrieval results start accumulating — is exactly the trajectory `MAX_ITERATIONS = 6` was chosen to interrupt before it becomes a problem.

A worked projection is now straightforward. This run's own growth rate, unmitigated by compression, would put a run at 4,491 tokens after 3 iterations. `compress_context` (Chapter 22B) replaces the raw retrieval messages with a single formatted context block precisely to break that trajectory — the reason it is a mandatory, not optional, phase transition in the state machine (Chapter 16.3) rather than something the LLM can simply decline to call.

23.4 How tool results inflate context fast

The single biggest contributor to context growth in this pipeline is not the system prompt or the conversation turns — it is retrieved chunk content arriving as raw tool results. ADR-015 fixed chunk size at 1,000 characters with 200-character overlap, which is roughly 250 tokens per chunk using the conventional chars/4 estimate this project's own `[CONTEXT SIZE @ iter N]` logging block uses for its quick estimate alongside the API's actual reported figure.

A single `retrieve_documents` call at `top_k=5` returns five chunks. Five chunks at roughly 250 tokens each is approximately 1,250 tokens of raw chunk text — before the source tags, formatting, and tool-call wrapper JSON that accompany every retrieval result are even counted. `_PROCESS_INSTRUCTIONS` allows 2-3 retrieval calls before compression, plus up to `MAX_TOTAL_RETRIEVALS = 5` across a run if the agent loops back — meaning an uncompressed run can accumulate 5,000-6,000 tokens of raw retrieved text alone, on top of everything else in the window.

> **Common pitfall — Counting queries, not tokens**
> A retrieval budget expressed only as "5 calls maximum" hides the real cost. Five calls at top_k=10 is twice the token load of five calls at top_k=5 for the identical iteration count — the call cap and the per-call top_k both belong in the same budget conversation, not just the call cap alone.

23.5 Reading real numbers — the total_prompt_tokens and total_completion_tokens log lines

`agent_query.py` accumulates two running counters across the entire `run_agent()` call: `total_prompt_tokens` and `total_completion_tokens`, both initialized to zero and incremented after every LLM response by reading `usage.get("prompt_tokens", 0)` and `usage.get("completion_tokens", 0)` from the API's own usage block — not an estimate, the provider's actual billed count. Both counters are threaded through every return path of the function, including early exits, so a caller always receives an accurate total regardless of which phase the run terminated in.

```
total_prompt_tokens     = 0
total_completion_tokens = 0
...
total_prompt_tokens     += usage.get("prompt_tokens", 0)
total_completion_tokens += usage.get("completion_tokens", 0)
```

This distinction — per-call tokens from `[CTXSIZE]` versus run-total tokens from `total_prompt_tokens` — answers two different questions. Per-call figures answer "did this specific request fit the window and how close was it to the ceiling." The run-total figures answer "what did this entire user-facing answer actually cost," which is the number that scales with usage volume and the one worth watching if a deployment's Groq bill or rate-limit headroom becomes a concern.

23.6 The reserved-output problem

Every LLM call in this pipeline implicitly reserves space for its response before it is sent, whether or not the caller thinks about it explicitly. A `max_tokens` parameter set too high does not just risk hitting the model's output ceiling — it silently shrinks the *input* budget available to that same call, because provider APIs typically require input plus requested output to fit within the total window.

This is why `_PROCESS_INSTRUCTIONS`'s "Max 400 words" constraint on the final answer (Chapter 14.9) is not purely a presentation rule. Four hundred words is roughly 550-600 tokens — small enough that reserving output space for it costs almost nothing against a 120,000-token usable input budget, versus reserving room for the model's full 8,192-token output ceiling on every call as an unexamined default.

23.7 Budgeting for the judge and merge LLM calls, not just the agent's

ADR-018's three-instance design — separate `llm`, `merge_llm`, and `judge_llm` objects — means a single user query's true token cost is not one context window's worth of consumption, but potentially several: the main agent loop's iterations, plus every NAC merge call, every DC redundancy-judge call, every LBC compression call, and the final grounding-judge call, each hitting its own context window independently.

None of these secondary calls carry anywhere near the agent loop's accumulated history — a merge call sees only the handful of chunks being merged, not the full conversation — but at scale, across many concurrent users or a high-volume deployment, the aggregate token spend across all three LLM roles is the number that determines actual operating cost, not the agent loop's `total_prompt_tokens` figure in isolation. A budget review that only inspects `run_agent()`'s own counters is looking at one instance out of three.

The three-instance design also means the *shape* of token spend differs by role in a way a single aggregate number would hide. The main `llm`'s calls grow across a session as history accumulates, matching the pattern Figure 23.2 traces. `merge_llm` and `judge_llm` calls, by contrast, stay roughly flat in size call to call — a merge call's prompt size depends on how many near-duplicate chunks it is merging, not on how far into the run the session has progressed. A deployment tracking cost trends over time should expect the agent loop's contribution to rise with query complexity while the compression and judging contribution stays comparatively stable, and a budget dashboard that conflates the two into one number obscures which part of the pipeline is actually driving a cost spike.

23.8 A token-budget checklist for a new agentic pipeline

- Know the model's total window AND its output ceiling — subtract the second from the first before calling the remainder "usable."
- Log both an estimated (chars/4) and an actual (API-reported) token count per call — they diverge, and the divergence itself is informative.
- Track a cumulative counter across the whole run, not just per-call figures — rate limits are usually per-minute totals, not per-call ceilings.
- Budget retrieval by tokens-per-call (top_k × chunk size), not call count alone.
- Count every LLM role in the pipeline — agent, merge, judge — not just the one issuing the user-facing answer.
- Cap output length explicitly wherever the answer format allows it — a small `max_tokens` reservation buys back real input budget.

Figure 23.1 lays the 8B model's window out as a labeled budget, and Figure 23.2 traces that budget being spent, call by call, in a real trace.

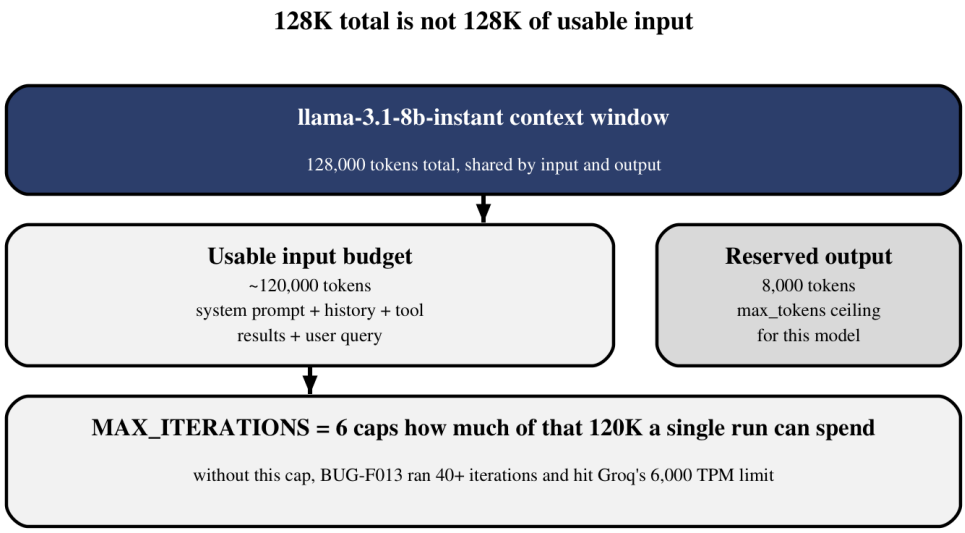

Figure 23.1 — The 128K total window splits into a reserved output ceiling and a usable input budget, and MAX_ITERATIONS exists to keep a run inside it.

Figure 23.2 makes the abstract growth rate from Section 23.3 concrete: three real iterations, three real cumulative totals, climbing toward the ceiling Figure 23.1 already named.

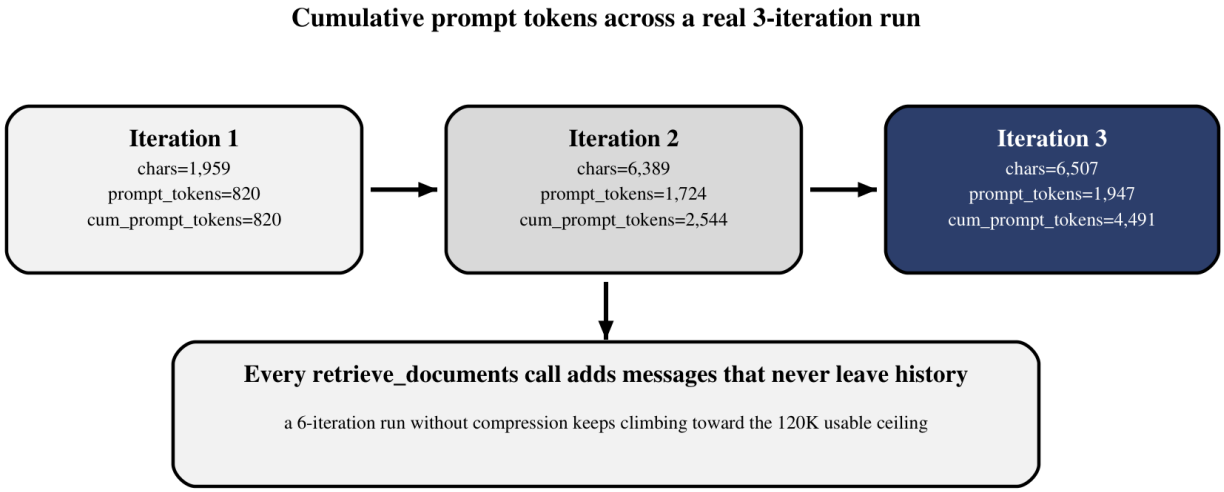

Figure 23.2 — Cumulative prompt tokens from a real trace, iteration by iteration, before compression intervenes.

A token budget is not an optimization to bolt on after a pipeline works — it is the reason `MAX_ITERATIONS`, `MAX_TOTAL_RETRIEVALS`, and the compression pipeline exist at all, discovered the hard way when BUG-F013's unguarded loop hit Groq's rate limit at 40-plus iterations. The next chapter asks a related but distinct question: even when a prompt technically fits inside the window, does the model still use all of it well, or does quality quietly degrade long before the token counter actually runs out?